Flexbox en grid in twintig minuten

BIJ labo 4 / hoofdstuk 4

DUUR 20 min klassikaal, daarna aan de opdrachten

Jullie krijgen dit labo een dag vóór de theorie. Dit document geeft je net genoeg om aan de vijf nabouwen te beginnen; de volledige uitleg staat in hoofdstuk 4 van de syllabus (de sectienummers verwijzen daarnaar); lees het deze week alsnog.

1. Container en items (4.1)

Tot nu vielen je boxen onder elkaar, in de volgorde van je HTML. Zet je op een element `display: flex` of `display: grid`, dan wordt dat element een **layoutcontainer** en worden zijn **directe kinderen** items die volgens nieuwe regels geschikt worden. Twee dingen onthoud je meteen. **De container organiseert, de items ondergaan**: "maak een rij" of "maak drie kolommen" schrijf je op de ouder, niet op de kinderen. En alleen directe kinderen doen mee: een vergeten wrapper-div tussen container en kaarten breekt de hele layout (F4.1). De witruimte-spatie en de baseline van inline-block uit labo 3 zijn hier verdwenen; dat geknutsel is voorbij.

2. Eén as of twee assen? (4.2)

Voor je typt, stel je bij elke layout dezelfde vraag: **verdeel ik langs één as, of span ik een raster over twee?** Eén as (een rij of een kolom, en de vraag is hoe de ruimte langs die lijn verdeeld wordt) is **flexbox**: een navigatiebalk, een knoppenrij, drie kolommen naast elkaar. Twee assen (rijen én kolommen moeten tegelijk kloppen) is **grid**: een fotorooster, een schilderij van Mondriaan, het grondplan van een pagina. **Vuistregel:** grid voor de pagina, flexbox voor de rij.

3. Flexbox: de rij (4.3)

```
.menu {
  display: flex;
  gap: 1rem;
  justify-content: space-between;  /* along the main axis   */
  align-items: center;             /* along the cross axis  */
}
```

De items staan op de **hoofdas** (standaard horizontaal); haaks daarop staat de **kruisas**. `justify-content` verdeelt langs de hoofdas: `flex-start`, `center`, `flex-end`, `space-between`. `align-items` lijnt uit op de kruisas: `stretch` (standaard, daarom hebben items in een rij vanzelf gelijke hoogte), `center`, `flex-start`. Met `flex-direction: column` wordt de hoofdas verticaal en draaien beide woorden mee. Tussenruimte maak je met `gap`, nooit met marges op de items: één waarde, alleen tússen de items, niet aan de randen. Een lijst als rij: `display: flex` op de `ul`, en de user-agent-bagage (bolletjes, insprong) zet je zelf uit.

4. Groeien: flex op de items (4.4)

```
.kolommen > * {
  flex: 1;   /* each child takes an equal share of the row */
}
```

`flex` is de ene property die op een **item** staat. `flex: 1` op alle kinderen: gelijke delen. `flex: 2` op één kind: een dubbel aandeel. `flex: 1` op één enkel kind: dat kind slorpt alle restruimte op. Dat laatste werkt ook in een kolom: in een kaart die als `flex-direction: column` is opgebouwd, krijgt de tekst `flex: 1`, en de knop eronder staat in elke kaart onderaan, hoe lang de tekst ook is.

5. Grid: het raster (4.5)

```
.galerij {
  display: grid;
  grid-template-columns: repeat(3, 1fr);   /* three equal columns */
  gap: 1.5rem;
}
```

`grid-template-columns` beschrijft de kolommen, één waarde per kolom. De eenheid `fr` is een deel van de beschikbare ruimte: `1fr 1fr 1fr` zijn drie gelijke kolommen, `2fr 1fr` twee derde tegen één derde, `12rem 1fr` een vaste zijbalk naast de rest. Gebruik fr en geen procenten: fr rekent de gaps mee, procenten niet (F4.4). Rijen hoef je meestal niet te beschrijven: de items vullen het raster in DOM-volgorde en de browser maakt vanzelf rijen, zo hoog als hun inhoud. Wil je wél vaste rijen (een raster dat het venster vult, een trap), dan `grid-template-rows: repeat(4, 1fr)` plus een hoogte op de container, bijvoorbeeld `100vh`.

6. Plaatsen op lijnen (4.6)

```
.uitgelicht {
  grid-column: 1 / 3;   /* from line 1 to line 3: two columns wide */
  grid-row: 2;          /* the second row                          */
}
```

Grid telt in **lijnen**, niet in cellen: drie kolommen hebben vier kolomlijnen, genummerd vanaf 1, en `-1` is de laatste. Een item plaats je van lijn tot lijn met `grid-column` en `grid-row`; `span 2` betekent "twee sporen breed, waar ik ook sta". Wat je niet plaatst, vult vanzelf verder; cellen die niemand opeist, blijven leeg. Dat is hoe je een Mondriaan of een trap bouwt.

7. Centreren en het grondplan (4.7)

```
body {
  display: grid;
  grid-template-rows: auto 1fr auto;   /* header, main, footer */
  min-height: 100vh;
}
```

Grid gebruikt dezelfde uitlijningswoorden als flexbox: `place-items: center` op een container zet alles wat erin zit in het absolute midden, horizontaal én verticaal. En het grondplan hierboven is de **sticky footer**: de header en de footer worden zo hoog als hun inhoud, `main` krijgt met `1fr` alle resterende ruimte, en `min-height: 100vh` (geen `height`, 3.8) vult minstens het scherm. De footer staat onderaan het scherm bij weinig inhoud en onderaan het document bij veel.

8. Je waarheidsinstrument: badge en overlay (4.9)

In de Elements-tab draagt elke layoutcontainer een klikbare badge, **flex** of **grid**. Eén klik tekent de overlay over de pagina: bij grid de lijnen mét hun nummers en de gaps, bij flex de items en hun verdeling. Twee vragen bij elke layoutbug: is dit element wel de container die ik denk (geen badge, geen container: je `display` zit op het verkeerde element), en zijn dít de items die ik denk (een vergeten wrapper omlijnt de overlay als één groot item)?

Aan de slag

Oefening 1 is punt 3 en 4 (een flex-rij, `flex: 1` en `flex: 2`, het cijfer centreren met punt 7 op het item zelf), oefening 2 is punt 3 (twee flex-containers: de balk en de lijst), oefening 3 en 4 zijn punt 5 en 6 (een raster beschrijven, dan plaatsen op lijnen), oefening 5 is alles samen in

het grondplan van punt 7. Stel bij elke nabouw eerst de vraag van punt 2, schrijf dan de container, dan pas de items. En klik de badge aan vóór je gaat zoeken.